VALIDUS

# Case Study
# Implementing a Trade Approval Process

## *Case Study: Implementing a Trade Approval Process*

Our company is developing a trade approval system where a user can submit trade details for approval and follow a structured workflow until the trade is executed or cancelled. This system will streamline and standardise the trade approval process for financial instruments such as forward contracts.

## Workflow Design

The trade approval process will support the following actions:

| Action | Explanation | Inputs |
|---|---|---|
| **Submit** | Submit a new trade request. | User ID (Requester), trade details (described below). |
| **Accept/Approve** | Approve a submitted trade request. | User ID (Approver). |
| **Cancel** | Cancel the trade request. | User ID (Requester or Approver). |
| **Update** | Update trade details (requires reapproval). | User ID (Approver), updated trade details. |
| **SendToExecute** | Send the approved trade to the counterparty for execution. | User ID (Approver). |
| **Book** | Book the trade into the system once executed by the counterparty. | User ID (Requester or Approver), confirmation of execution. |

Trade details: Each trade will include the following information:

| Field | Description |
|---|---|
| **Trading Entity** | The legal entity conducting the trade. |
| **Counterparty** | The entity on the other side of the trade. |
| **Direction** | Specifies whether the trade is a "Buy" or "Sell". |
| **Style** | Assumes the trade is a forward contract. |
| **Notional Currency** | Currency of the notional amount (e.g., EUR, GBP, USD). The full list is available at IBAN Currency Codes. |
| **Notional Amount** | The size of the trade in the selected notional currency. |
| **Underlying** | A combination of eligible notional currencies. The notional currency selected must be part of the underlying. |
| **Trade Date** | The date when the trade is initiated. |
| **Value Date** | The date when the trade value is realized. |
| **Delivery Date** | The date when the trade assets are delivered. |
| **Strike** | Agreed rate. This information is only available after trades are executed. |

**Validation Rule**: Trade Date ≤ Value Date ≤ Delivery Date.

## State Transitions

The actions transition the trade between the following states:

| State | Description | Allowed Next Actions |
|---|---|---|
| **Draft** | The trade has been created but not submitted. | Submit |
| **PendingApproval** | The trade has been submitted and is awaiting approval. | Accept, Cancel |
| **NeedsReapproval** | The trade details were updated by the approver, requiring reapproval from the original requester. | Approve, Cancel |
| **Approved** | The trade has been approved and is ready to send to the counterparty. | SendToExecute, Cancel |

| | | |
|---|---|---|
| **SentToCounterparty** | The trade has been sent to the counterparty for execution. | Book, Cancel |
| **Executed** | The trade has been executed and booked. | None (end state). |
| **Cancelled** | The trade has been cancelled. | None (end state). |

## Requirements

You are asked to implement a library with a suitable API to support this workflow. The public API should expose the core actions described above to drive state changes. Data can be stored in memory as this is a prototype.

The API must:

1. Written in Python or Rust.
2. Enforce validation rules for trade details.
3. Prevent invalid state transitions and unauthorized actions.
4. Allow users to view the history of a trade, including:
    - Trade details at any previous state.
    - A tabular history of actions with user IDs, timestamps, and the state transitions.
    - Differences between two versions of trade details (e.g., updates to notional amounts).
5. Allow trades to be sent to a counterparty and marked as executed.

The use of ChatGPT or any generative AI tools is acceptable for the case study; however, you must disclose the usage. If you choose to utilise generative AI, please explain your specific contributions or enhancements to the work.

Additionally, we encourage you to think beyond the given requirements and consider what extra value or features you could add to the case study.

## Example Scenarios

*1. Submitting and Approving a Trade*

**Scenario**: A user submits a trade for approval, and it is approved.

| Step | Action | User ID | State Before | State After | Notes |
|---|---|---|---|---|---|
| **1** | Submit | User1 | Draft | PendingApproval | Trade details provided. |
| **2** | Approve | User2 | PendingApproval | Approved | Approver confirms trade. |

*2. Updating a Trade Detail*

**Scenario**: An approver updates the trade details, requiring reapproval.

| Step | Action | User ID | State Before | State After | Notes |
|---|---|---|---|---|---|
| **1** | Submit | User1 | Draft | PendingApproval | Trade details provided. |
| **2** | Update | User2 | PendingApproval | NeedsReapproval | Approver updates the notional amount. |
| **3** | Approve | User1 | NeedsReapproval | Approved | Requester reapproves updated trade details. |

*3. Execution*

**Scenario**: An approved trade is sent to the counterparty and marked as executed.

| Step | Action | User ID | State Before | State After | Notes |
|---|---|---|---|---|---|

| | | | | | |
|---|---|---|---|---|---|
| **1** | Submit | User1 | Draft | PendingApproval | Trade details provided. |
| **2** | Approve | User2 | PendingApproval | Approved | Approver confirms trade. |
| **3** | SendToExecute | User2 | Approved | SentToCounterparty | Trade sent to counterparty. |
| **4** | Book | User1 | SentToCounterparty | Executed | Trade executed and booked. |

*4. Viewing History and Differences*

- **History API**: Return a table of all actions with details.
- **Differences API**: Show changes between versions, e.g., {"notionalAmount": ("1,000,000", "1,200,000")}.

## Deliverables

- Implement the API for the described workflow in Python or Rust.
- Include unit tests to validate key actions, state transitions, and validations.
- Provide documentation for the API and example usage scenarios.